

A Lightweight Distributed Security Protocol with Adaptative Key Management for Programmable Matter Based on Modular Robots

Yousseu Faye
LI3/Department of Computer
Sciences
University Assane Seck of
Ziguinchor
I Ziguinchor, Senegal
yfaye@univ-zig.sn

Abdallah Makhoul
Femto-st/Computer Science
Department- DISC
University Marie et Louis
Pasteur
Montbeliard, France
abdallah.makhoul@univ-fcomte.fr

Serigne Mbacke Diene
LI3/Department of Computer
Science
University Assane Seck of
Ziguinchor
Ziguinchor, Senegal
s.diene20150547@zig.univ.sn

Mohammed Ouzzif
C3S /ESTC
Hassane II University of
Casablanca
Casablanca, Morocco
ouzzif@gmail.com

Abstract—Programmable matter can change its physical properties and, more likely, its shape, depending on internal or external action. Here, we approach programmable matter in the context of micro-robots, millimeter-sized mobile microcomputers that can move around each other, communicate, change color and hook onto other micro-robots to form different shapes. The current challenges are both hardware and software. The main research challenges are diverse and varied, and focus on self-configuration, clustering, synchronization time, decision-making, analysis of detected data and security. To date, very little work has been done in the field of security, and this does not take into account possible developments relating to the material itself, whose physical configuration underpins exchange and self-reconfiguration protocols. In this paper, we propose a solution that takes current security concerns into account, and which may be able to adapt to a future evolution of programmable matter based on modular robots. This solution is based on resource-saving cryptographic mechanisms to implement a robust authentication solution that underpins a flexible key management mechanism coupled with a confidential data exchange of the structure thus formed.

Keywords—Modular robot, programmable matter, Lightweight cryptography.

I. INTRODUCTION

Modular robots are robots designed with modules that connect together to form autonomous systems with variable morphology that can perform different functions or tasks. One of the potential applications of self-reconfigurable modular robots is programmable matter [1]. Programmable matter based on modular robots envisages materials constructed as an assembly of thousands of millimeter-sized, micro-scale robotic modules that can move around each other, communicate, change color and hook onto other micro-robots to give different shapes [2]. Application areas are diverse and include space shuttles, medical devices, search and rescue, etc. Whereas conventional robots can only adapt to a single application or purpose, modular robots can fulfill a wide range of missions [3]. Today, challenges are both hardware and software. The main research challenges

are more oriented towards self-configuration, clustering, synchronization time, decision-making, analysis of sensed data and safety [3, 4, 5, 6]. Up to now, work on security has not been widely addressed, and to the best of our knowledge, only one work exists, and this focuses more on the current hardware and software configuration of the amoebot model [7] to propose a multi-stage authentication service without taking into account possible evolutions relating to the material itself, which through its physical configuration underpins the exchange and autoconfiguration protocols [8]. In this paper, we propose a solution that takes into account current security concerns, and which could possibly be adapted to a future evolution of modular robot-based programmable matter. This solution has the advantage of using lightweight cryptographic techniques such as hash functions, XOR operation, synchronization time and nonces to implement a robust authentication mechanism that supports a flexible and scalable key management protocol through which communication confidentiality is ensured. After analyzing the security properties, we show that authentication, key management and confidentiality are ensured during autoconfiguration, and after structure formation. The key management protocol is adapted according to the network's operating architecture (i.e. coordination of the structure by a computer, or a leader module), the autoconfiguration process (serial by one module or parallel by several modules in clustering), the degree of connectivity and common key sharing in the same neighborhood.

The rest of the paper is organized as follows. We set the context in section 2 and study background in section 3. Section 4 presents our hypotheses and the operating architecture of the modular robot system. Security challenges are presented in Section 5 and the proposed solution in Section 6, followed by an analysis in Section 7. The conclusion and outlook are presented in Section 8.

II. CONTEXT

The classification of modular robotic systems is quite a difficult task, due to the number of existing projects and the diversity of

their task, form, size or control system. Some modules are controlled by external computers. Others are directed by a control module or are completely autonomous. Nevertheless, all modules are equipped in the same way, both physically and logically. A security solution is highly dependent on system architecture and configuration when deployed on the lower layers. A security solution for programmable material must be contextualized according to its framework and application domain, which is strongly correlated to its architecture. This application framework depends on the physical nature of the programmable material. For example, we can distinguish between programmable materials based on modular robots such as ATRON and Sambot, and programmable materials made up of tiny cubic modules such as BlinkyBlocks (4cm edge), cylindrical modules such as 2D Catoms (6mm long) or spherical modules such as 3D Catoms (millimetric scale). Each material is characterized by a number of properties.

- *its mobility*: is ensured by an actuating mechanism; BlinkyBlock is static, while 2D Catoms and 3D Catoms are mobile.
- *its degree of freedom of movement*: 2D Catoms move by rolling over their fixed neighbors, while 3D Catoms move around their neighbors.
- *its number of neighbors*: BlinkyBlock can have up to 4 neighbors, 2D Catoms up to 6, 3D Catoms up to 12.
- *its field of application*: the BlinkyBlock is used to create interactive works of art, education and research on distributed systems.

In the first case, where self-reconfiguring modular robots can modify their general form to adapt to the environment or to a task or scenario, depending on the data detected and analyzed from the real environment in which they evolve. This can be referred to as a modular robot system. In the second case, this set of tiny modules or particles forms a distributed system where each module has communication and computing capacity. Modules can be externally controlled by a computer, controlled by a leader module, or operate autonomously by executing a distributed program to switch from one form to another. This is known as modular robot-based programmable matter. In this paper, we focus on programmable matter based on modular robots.

III. BACKGROUND

A. Programmable matter vs WSN and IoT

Modular robots have similarities with Wireless Sensor Networks (WSN) and the Internet of Things (IoT), but also differences that have a major impact on security. Security mechanisms, resource-saving algorithms and protocols have been proposed for IoT, RFID and WSNs [9, 10, 11], but are not suitable due to the specificity of modular robot systems. Because of sensors on both IoT objects, wireless sensor networks (WSNs), and modular robot systems (SRMs) or programmable matter based on modular robots present security limitations, especially when it comes to executing traditional, resource-hungry security algorithms. Security algorithms proposed for WSNs and IoT would be suitable for sensors on modular robots

[12, 13, 14], but modular robot systems present environments and structuring that make it difficult to execute some algorithms. Differences with modular robots mean that some protocols and security mechanisms are unsuitable.

Differences with WSN

- A very large number of modules or particles or cells
- Particles are anonymous and have no unique identifier like sensors
- Limited number of neighbors
- Possibility of contact communication
- Highly dynamic topology during autoconfiguration
- Neighbor-to-neighbor communication: no routing

Differences with IoT

- - Particles are anonymous, with no unique identifier or IP address.
- No centralized control
- Modules are not connected to the Internet
- Possibility of contact communication

B. Security analysis in programmable matter

Security in modular robot-based programmable matter is a real challenge. Due to their characteristics, security protocols in WSN, IoT and RFID are inadequate. Security issues are not well addressed. To the best of our knowledge, only the Protocol of Lightweight Security Embedded in an Architecture of Nanobots (PROLISAN) has been proposed [7], and its scope of application is limited to a category of modules. PROLISAN uses lightweight cryptographic mechanisms based on hash functions and symmetrical block ciphers to reduce memory storage and computation time. It provides two security services: authentication of modules when they connect to the system, and confidentiality of communications between modules to prevent unauthorized third parties from learning about system behavior and decision-making. Four versions are available, with different levels of security and authentication.

The first version provides a weak authentication service based on the fingerprint from the hash of the source code. It does not guarantee membership in the structure, replay old messages is possible because the same fingerprint will be used by all modules. We only need the source code to be able to create an authentication message.

The second version provides an authentication and confidentiality based on hashing the code into two fingerprints: one is used for authentication and the other as a data encryption key for confidentiality. Just like the first version, it does not guarantee membership in the structure, replay old messages is always possible because the same fingerprint will be used by all modules. In addition, it requires more resources with two fingerprints. Unlike the first, its security is designed based on the hash of the first n lines for authentication and the last m ($n \neq m$) lines for key. This does not solve the problem highlighted in the first version since the hash function used will always

produce the same size of fingerprints and the same code will always produce the same fingerprint. If the channel is not physically secure, it is possible to capture a fingerprint to authenticate or decrypt the messages. So that it can work, communication must be by contact, the signal must not be accessible to neighbors, and the code must not be known to the general public.

The third version redefines authentication. For both authentication and confidentiality, it uses the hash of a line instead of a part of the source code as in version 2. Its security depends on the line generation mechanism based on the authentication number and a chaotic function to determine the lines to be hashed. However, even if the character of determining the line to be hashed is more or less random, the hash of the line is sent for authentication, so replay attack is always possible. If the chaotic function is known, it is necessary to have the source code to create an authentication message. In version 2, the same key was used by neighbors, while here we have a key for each pair of modules. This version is based on the fact that malicious nodes do not know the protocol, which is a limitation for public use. Furthermore, the authentication mechanism stipulates that the moving module authenticates itself to the fixed module, there is a problem when two modules are moving, each module can remain waiting (infinite idle listening).

The fourth version, an improvement on the third, solves the problem of module mobility by using module identities distributed by a leader. Thus, modules authenticate themselves through their ID, their Clock, which determines the part to be hashed. Module A authenticates B through ID_B, the hash of part of B's code $H(\text{Code}_B)$, and the Clock_B that B sent to it. And through B's clock, A identifies the corresponding part of its code, which it hashes and verifies with the code received from B. This version uses the same encryption method as version 3. Although it regulates module movement, this version runs in a more restrictive environment since a leader and IDs are required, distributed a priori. This raises the question of contact-based communication for moving modules. If the clocks were synchronized, there would be no need to send them; sending a clock simply reveals the part of the code to be used for authentication. This poses almost the same vulnerabilities as version 3.

IV. ASSUMPTIONS AND SYSTEM ARCHITECTURE

A. System assumptions

Here we plan to study the security of a programmable matter-based modular robot system in a more advanced and safety-restricted environment. The modular robot system studied is very similar to 3D Catoms and has the following properties:

- The system is distributed and controlled by a computer or a leader module whose role is simply to provide the signal and shape for self-configuration.
- A module communicates only with its neighbors through one of its connectors or a wireless signal.
- In the structure, a module has at least one neighbor.
- A module can have many neighbors as necessary.

- All modules are identical and have the same source code. There is no identifier for module
- The clocks are reset simultaneously and each time the modules receive the signal to start the self-configuration process.
- The security of the modular robot system is studied during formation or self-configuration phase, or when transitioning from one shape to another.
- The structure will be formed from a single node
- The structure is composed of a large number of modules, to the point that it is impossible to manually configure them a priori
- Self-configuration is possible in several forms, and can be done in a public environment. In this way, sharing secrets prior to deployment when designing module would only be useful for dedicated applications where the modules are built for limited or dedicated use and not for public use.

B. System architecture

In this section, we first describe programmable matter in a highly evolved context, a context that reveals the trend of programmable matter even if until now modular robot-based systems are limited in terms of basic architecture, movement and communication. Current hardware and software technologies operate on the basis of chain, lattice or hybrid architecture sometimes supported by a sandbox with modules that rotate around their pivot neighbors or slide on the lattice and can communicate by contact. For more success in the self-configuration process, more freedom is needed on the architectures, on the movements and communications of the modules. For a scalable security protocol that will take into account the future technological and software mutations of modular robot systems, we put ourselves in a context that takes into account the following assumptions:

- Modules that can communicate by contact or wireless
- Modules can move on a lattice or their neighbors, as well as on a support on which they are placed or in the environment in which they operate.
- Modules that can move in all directions with a complete degree of freedom...

We take into account these assumptions, and always remain in a framework that incorporates current realities. We can, in addition to current architectures, find a multitude of physical architectures, and it will be more a question of focusing on the functional or configuration architecture. As shown in Fig.1, we distinguish three types of configuration and operating architectures identified based on existing work: an architecture where the modules are controlled by a computer (Fig. 1.-a), an architecture where the modules are controlled by a leader module ((Fig. 1.-b), and another where the modules form a distributed system and are completely autonomous.

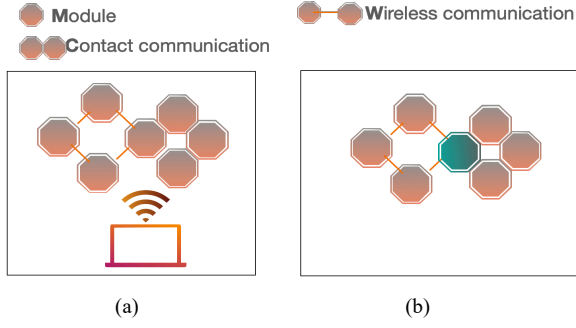


Fig. 1. Architecture

V. SECURITY CHALLENGES

The main challenge is related to the modules' limited resources, which make resource- and energy-intensive security mechanisms inadequate. In addition, the lack of identifiers, which complicates authentication mechanisms.

Some security solutions advocate the sharing of information before security configuration phase, like with access points or IoT networks, etc. For private use. However, for public use of programmable materials based on modular robots, it is virtually impossible to configure hundreds of thousands of modules, and the lack of identifiers makes filtering and address-based identification mechanisms difficult.

Programmable materials like 3D Catoms communicate by contact; the signal is not broadcast, which reduces communication vulnerabilities and the number of communication neighbors. However, for advanced programmable hardware, with fewer communication constraints, wireless and contact communication may be possible for greater freedom.

The use of contact communication would be an asset against channel attacks. However, we are considering a sufficiently autonomous scalable hardware capable of communicating in contact and by wireless channel.

In terms of authentication, the challenge is major since there is no central entity in the strict sense of the term like an entity that manages authentication. The question of authentication against what arises. Hence the difficulty of developing an authentication mechanism through a peer-to-peer relationship.

Confidentiality is ensured by encryption, which requires key management. The absence of a trusted third party makes the use of asymmetric cryptography difficult. And in the absence of a module identifier, sharing symmetric keys without third parties poses more challenges in a fully distributed environment.

Key management is a real challenge, in that it must be memory-, computationally-, and energy-efficient to meet the very limited resources of the modules. It must also be deployed in a distributed environment without secret sharing, making it difficult to implement a key management mechanism from scratch.

VI. THE AKC-PM PROTOCOL

We propose an Authentication, Key Management and Confidentiality protocol for Programmable Matter (AKC-PM)

based on modular robots. This protocol allows authentication at the end of which a peer or group key is established to implement confidentiality through encryption. We consider that the structure will be formed from a module at a given position. Thus, authentication will consist of a module proving its membership in the structure by proving that it executes the same code as those of the structure. Since modules have same code, and there is no trusted third party at the start, it is difficult for two modules to carry out authentication with a peer-to-peer relationship. Since the structure will be formed from a single module (initial module), we distinguish here three types of authentications illustrated in Fig. 2.

- Authentication of the initial module with the first module to form the structure (Fig. 2.-a)
- - Authentication of a module to join the structure (Fig. 2.-b)
- - Authentication of two modules already belonging to the structure (Fig. 2.-c)

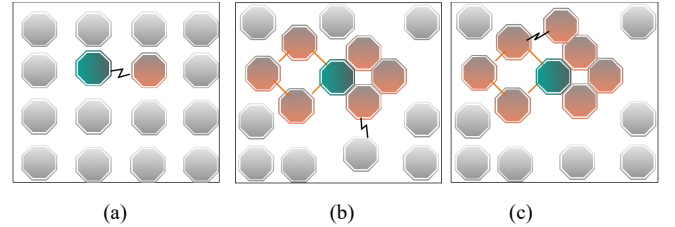


Fig. 2. Types of Authentication

The first version of the protocol uses a single key for the entire system, the second uses four keys for the entire system, and the third uses multiple keys that depend on the number of modules.

TABLE I. NOTATIONS

	<i>Table column subhead</i>
copy	More table copy ^a
Mi	One module of the structure
iM	Initial Module
nM	The Module n
vM	Neighbor
sM	All neighbors
ni	Number once i
Ts	Current sending time
Tr	Current reception time
Li	Ligne i in the code
Nb	Number of lines code
H()	One-way hash function
⊕	Exclusive-or (XOR) operation
	Concatenation
→	Transmission
Δt	Transmission delay
≈	Approximate value to nearest integer

A. Structure formation

A new module nM wishing to join the structure authenticates with it, and on completion of this authentication, a key is established with a module M_i in the structure, and the link M_i to M_n is considered descending and that nM to M_i ascending. There are two possible scenarios for forming the structure. The structure begins with the initial iM module, which authenticates the first module. The first module has to prove its authenticity to the iM .

- The structure starts with the initial iM module, which will connect the first module with which they authenticate each other.
- It is assumed that when the experiment is launched, the iM module is designated by its geographical position or by an election process.

When a module launches a connection, it specifies in the request the number of links it has in the structure, the period T_s for sending the request, a nonce n_i for both identifying the request and calculating the line to be hashed. The T_s period is the nearest integer value of the sending time noted t_s at clock level, divided by ten: $T_s \approx (t_s/10)$. This makes it possible to correct small clock desynchronization offsets.

- When the experiment is launched and the initial iM module is in place.
- The new module $1M$ sends an authentication request to the iM to prove that it belongs to the structure. $1M$ generates a nonce n_0 . An authentication request is identified by its composition, so there's no need to tell module iM that it's a first request for the formation of the structure, since it knows it's the initial module and isn't linked to any module. $1M$ calculates: $n_1 = H(n_0)$; the authentication fingerprint K_0 by applying the hash function $H()$ on line $L(n_0 \text{ modulo } Nb)$ of the source code, i.e. $K_0 = H(L(n_0 \text{ modulo } Nb))$; $x = T_s \oplus n_0$ to protect the nonce n_0 . then sends $1, n_1, x || K_0$ to module iM . The 1 in the request indicates that $1M$ has not yet joined the structure, and the n_1 in the request enables iM to check that the clocks are synchronized;
- After receiving the message, iM examines the message reception time T_s at clock level, calculates $T_s \approx ((Tr - \Delta t)/10)$, then calculates $n_0' = x \oplus T_s$, and checks whether $n_1 = H(n_0')$. If so, synchronization is good. iM must now authenticate $1M$ by checking whether $K_0 = H(L(n_0' \text{ modulo } Nb))$. If so, iM launches the key generation challenge by generating a nonce n_2 , calculates $x_1 = n_2 \oplus n_0$, generates the key $K_1 = H(L(n_2 \text{ modulo } Nb))$ and sends x_1 to $1M$.
- nM calculates $n_2 = x_1 \oplus n_0$, and in turn generates the key $K_1 = H(L(n_2 \text{ modulo } Nb))$. Thus iM and $1M$ have authenticated each other and share the key K_1 , as shown in Algorithm 1.

Algorithm 1: Authentication for structure formation

SF-1 : iM	Chooses n_0 ,
: iM	Computes $n_1 = H(n_0)$
: iM	Computes $K_0 = H(L(n_0 \text{ modulo } Nb))$
: iM	Computes $x = T_s \oplus n_0$
SF-2 : $1M \rightarrow iM$	$1, n_1, (x K_0)$
SF-3 : iM	Computes $T_s \approx ((Tr - \Delta t)/10)$
: iM	Drift $n_0' = x \oplus T_s$
: iM	Verify if $n_1 = H(n_0')$
: iM	Verify if $K_0 = H(L(n_0' \text{ modulo } Nb))$
SF-4 : iM	Chooses n_2
: iM	Computes $x_1 = n_2 \oplus n_0$
: iM	Computes $K_1 = H(L(n_2 \text{ modulo } Nb))$
: $iM \rightarrow 1M$	x_1
SF-5 : $1M$	Computes $n_2 = x_1 \oplus n_0$
: $1M$	Computes $K_1 = H(L(n_2 \text{ modulo } Nb))$

B. Authenticating a new module to the structure

In the same way, the protocol takes place between $2M$ and iM , except that at phase SF-1, the nonces n_0 and n_1 chosen by $2M$ will be different from those of $1M$. However, iM always keeps its nonce n_2 at phase SF-4, so that it can generate the same key as with K_1 . This gives the key distribution in Fig. 3.a) To have a unique key in the network, the modules already linked $1M$, $2M$, $3M$ and $4M$ will play the same role as the initial module (iM) with regard to the modules not yet linked, leading to the key distribution in Fig. 3.-b).

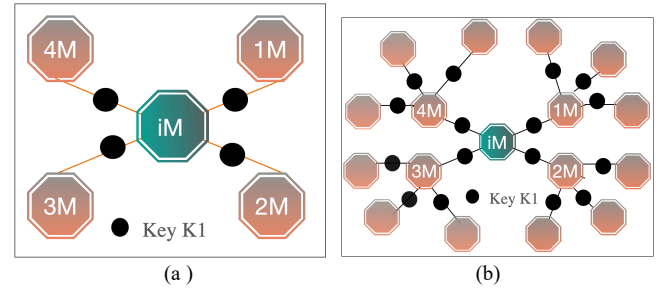


Fig. 3. : Single-key model

From Algorithm 1, we can move towards a scheme with 4 keys in the structure. In this case, the protocol is the same as that between iM and modules $1M$, $2M$, $3M$ and $4M$. However, for key generation, $2M$ applies the hash function 2 times, $3M$ applies it 3 times and $4M$ applies it 4 times to generate the keys $K_2 = H(H(K_1))$, $K_3 = H(H(K_2))$ and $K_4 = H(K_1)$. To find out the number of times to apply $H()$, iM indicates it by a number in the message transmitted to phase SF-4, or in a protocol fashion calculates: $x_1 = (n_2 || n_2) \oplus (n_0 || n_0)$ for $1M$; $(H(n_2) || n_2) \oplus (n_0 || n_0)$ for $2M$; $(H(H(n_2)) || n_2) \oplus (n_0 || n_0)$ for $3M$ and $(H(H(H(n_2))) || n_2) \oplus (n_0 || n_0)$ for $4M$. The nM modules, having added their n_0 , need to extract n_2 and see how many times to apply $H()$ to find $H((n_2))$, which is the remainder of the concatenation. Then we'll hash one more than the number of hashes made to find equality. In the same way, $1M$, $2M$, $3M$ and $4M$ connect their neighbors. This is shown in Fig. 4.

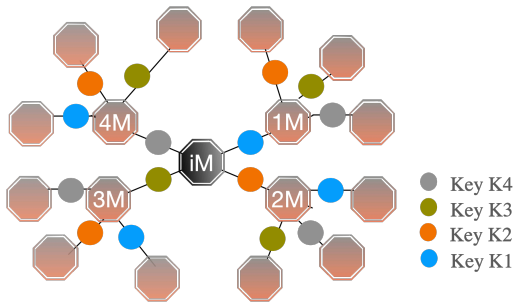


Fig. 4. 4-key model

We can have a scheme that uses less hashing and creates new keys as the structure grows. In this case, the protocol takes place between 2M and iM as in Algorithm 1, except that in phase SF-4 of the algorithm, iM chooses K1 instead of n2, calculates $x1 = K1 \oplus n0$ and $K2 = H(L(K1 \text{ modulo } Nb))$. The protocol continues between iM and modules 3M and 4M, respectively generating $x1 = K2 \oplus n0$ with $K3 = H(L(K2 \text{ modulo } Nb))$ for 3M and $x1 = K3 \oplus n0$ with $K4 = H(L(K3 \text{ modulo } Nb))$ for 4M. module iM shares 4 distinct keys with 1M, 2M, 3M and 4M. Module 2M can continue the protocol with 5M by generating $x1 = K2 \oplus n0$ with $K3 = H(L(K2 \text{ modulo } Nb))$ as shown in Fig. 5..

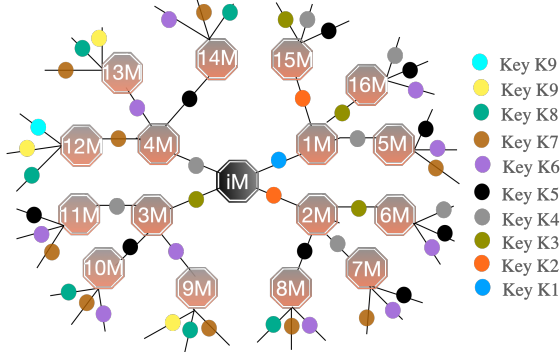


Fig. 5. Multi-key model

This method allows you to generate a wide range of key usage models, from a single key for the entire network, to a key for each pair of modules, to a key for each group of modules. And the choice of key usage model can be based on structural form and degree of connectivity.

C. Authentication of a structure module

Now, we'll look at the connection of a module already linked to the structure with another already linked. Remember that a structure module must not connect to a module that is not part of the structure. With the exception of the initial iM module, any module that receives a connection attempt without being linked to the structure rejects it. In the case of a module in the structure that is formed from two modules, any module that receives a connection attempt without being linked to the structure sends a connection request in return, so that authentication is mutual. A module that is already connected sends a proof of membership to the module to which it is connected and to the module to which it wishes to connect. Here we give the example of Fig. 6. - (a) where module 1M wishes to connect to module 2M with iM the neighbor to which it is linked to the structure with key K1.

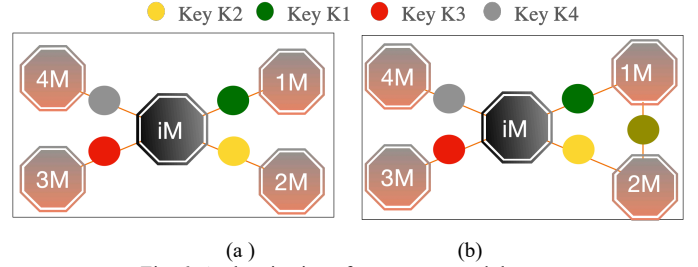


Fig. 6. Authentication of a structure module

Module 1M chooses its $n0$ and calculates $n1 = H(n0)$ and $x = K1 \oplus n0$ then sends $(2, x)$ to its neighbor iM and $(2, n1)$ to 2M. The 2 in the request $(2, x)$ indicates that 2M already has a link in the structure, and the 2 in the request $(2, n1)$ indicates to iM that this is a message to be propagated at 2 hops by decrementing, and when $(0, n1)$ is received, the message is no longer to be propagated. 2M, after receiving the message from iM, derives $n0$ and if $H(n0) = n1$, does not propagate the message received from iM, as it has just authenticated 1M. 2M proceeds to key generation from phase SF-5 of Algorithm 2 where it chooses K2 to be able to generate key K3 as shown in Fig. 6.- (b). The other neighbors of iM, for each of their neighbors sM, calculate $xv' = n0 \oplus Kv$ with Kv the neighbor's key.

Algorithm 2: Authentication by proof of membership

SF-1 : iM	Chooses $n0$,
: iM	Computes $n1 = H(n0)$
: iM	Computes $x = K1 \oplus n0$
SF-2 : iM $\rightarrow$ iM	$(2, x)$
: iM $\rightarrow$ 2M	$(2, n1)$
SF-3 : iM	Drift $n0 = K1 \oplus x$
: iM	Computes $xv = n0 \oplus Kv // Kv = K2$
: iM $\rightarrow$ sM	1, xv
: vM $\rightarrow$ sM	0, xv
: 2M	Drift $n0 = K2 \oplus x$
: 2M	Verify if $n1 = H(n0)$
SF-5 : 2M	Chooses K2
: 2M	Computes $x1 = K2 \oplus n0$
: 2M	Computes $K3 = H(L(K2 \text{ modulo } Nb))$
: 2M $\rightarrow$ iM	$x1$
SF-6 : iM	Computes $K2 = x1 \oplus n0$
: iM	Computes $K3 = H(L(K2 \text{ modulo } Nb))$

VII. SECURITY ANALYSIS

A. Security

In this section, we provide a security analysis of services provided and a theoretical evaluation of performance and robustness against attacks. The most difficult thing in this environment, and indeed in any distributed environment without a secret channel or trusted third party, is to share a key without any shared information. Here, we use clock synchronization at the beginning and only at the beginning of each autoconfiguration process, when the clocks are well synchronized. We use an approximation to eliminate any offset with some degree of uncertainty. Calculating the transmission clock divided by 10 and rounded off to the nearest integer allows us to correct for uncertainties of -5 to +5 of the normal value.

For example, for an offset of 8 units between the receiver and transmitter clocks (i.e. 96 for the receiver and 104 for the transmitter), the receiver finds $Ts = \frac{96}{10} = 9,6$ or 10, which is also equivalent to the approximate value of the transmitter clock, $Ts = \frac{10,4}{10} = 10,4$ or 10.

In Algorithm 1, the nonce $n0$ used is never disclosed and enables a new module to authenticate itself and generate the key that will be shared between iM and nM , while the fingerprint $n1$ derived from $n0$ will be used to verify synchronization and identify the request. If phase SF3 of Algorithm 1 is successfully completed, iM will have authenticated $1M$ by proving that it has the same code. In phase SF-5, iM challenges $1M$ through a generation process during which $1M$ is asked to prove by another line that it has the same code as iM , otherwise it will not be able to share a common key with iM . If $1M$ succeeds in generating the expected key, it will also have mutually authenticated iM .

The replay attack, which is the most critical, is impossible for several reasons. A repeated message will never be synchronized, as it will be delayed and the Ts calculation will fail. Without $n0$, the message sent in phase SF-2 of Algorithm 1 cannot be generated. In addition, the protocol offers a wide range of key generation methods, from one key shared by the entire network to one key per module pair. It should be noted that for a unique key, the number of hash operations required to generate the key is greater. This makes it easier to authenticate two modules in the structure. On the other hand, as we move towards a pairwise key, the number of hash operations required to generate the keys decreases, the probability that two modules close to the structure will share a key becomes low, and their authentication process for a future connection will be less straightforward than for a single key. After the authentication and key generation phase, two neighbors will share at least one key and will be able to encrypt/decrypt data. It's important to note that keys don't need to be stored, but rather managed. All a module needs to store is the nonce $n0$, from which it can generate keys and possibly other nonces.

B. Overhead Cost Comparisons

We analyze and compare our proposed solution to show its complexity (computation, time, communication, storage) and its resistance to forgery and repetition attacks. Complexity is assessed for the formation of a secure link between two modules. Table II shows that AKC-MP offers better safety than all 4 PROLISEAN versions, but consumes slightly more energy than versions 1 and 2. If we quantify the time taken for each hash, encryption or transmission operation, we'll be able to determine how long the autoconfiguration process will take. This will depend on the total number of links in the structure, as Algorithm 1 is executed on each link. We can model this system with graphs, where the nodes are the modules, the degree of node will be its neighbors. We can then calculate the number of links to know how many times Algorithm 1 will be executed. If v is the number of neighbors for each module and m the total number of modules, then the number of links is: $w = \frac{v.m}{2}$. Any node wishing to join the structure executes Algorithm 1 in coordination with one of its neighbors, and applies Algorithm 2

when it initiates a request over other links or even re-execute Algorithm 1 instead of Algorithm 2. To form a structure in which each module has v neighbors, Algorithm 1 is executed m times, and Algorithm 2 or Algorithm 1 is executed $(\frac{v.m}{2} - m)$ times. As structures where each node is connected to all of its neighbors are virtually nonexistent, we will estimate the computation time of the proposed solutions as a function of the number of links. We can see from Table II the total overhead cost of the proposed scheme compared to the different versions of PROLISEAN. The $O(n)$ complexities for versions 3 and 4 are due to the chaotic function. The cost c_i is given by (1) for Algorithm 1 and by (2) for Algorithm 2 or (3) if Algorithm 1 is fully utilized. The total global cost is given by (3). So, for a structure to be formed, the overall time complexity will be linear with the number of links and will be $O(w)$.

$$C1 = m(6TH + 2Tx + 4XOR) \quad (1)$$

$$C2 = (\frac{v.m}{2} - m) (4TH + 4Tx + 6XOR) \quad (2)$$

$$C3 = (\frac{v.m}{2}) (6TH + 2Tx + 4XOR) \quad (3)$$

TABLE II. OVERHEAD COMPARISONS

Solutions » Parameters	AKC-MP	PROLISEAN			
		V1	V2	V3	V4
Space Complexity	O(1)	O(1)	O(1)	O(1)	O(1)
Time Complexity	O(1)	O(1)	O(1)	O(n)	O(n)
Storage Complexity	O(1)	O(1)	O(1)	O(1)	O(1)
Transmission Complexity	O(1)	O(1)	O(1)	O(1)	O(1)
Number of keys	Pairwise	No	1	Pairwise	Pairwise
Authentication	Hight	Medium	Medium	Medium	Hight
Confidentiality	Hight	No	Low	Low	Hight
Replay again replay Attacks	Hight	No	No	Low	Medium
Protection again Forgery Attacks	Hight	No	No	Low	Medium
Overhead Cost	6H+2Tx + 4XOR	2H+2Tx	4H+2Tx	2H+2Un + 4Tx	4H+4Tx

T_H : the time for performing a one-way hash function $h()$.

T_{XOR} : the time for performing an XOR operation.

T_x : the delay time for the communication taken place in one-hops

Un: Chaotic arithmetic function.

We performed a simulation using the characteristics of a very low resources processor, the ATtiny45 by the corporation ATMEL equipped with an 8 MHz processor. We consider a 128-lines code for the modules, with each line being 256 bits in size. We use the SPONGENT-160 hash function, which offers a good compromise between security and lightweight performance, and is suitable for microcontroller units. On a Ttiny45, the execution time of SPONGENT-160 is 1s for the entire code and 1.5ms on one line, that of XOR is 2,5 μ s on 160 bits, that of the chaotic function is 225 μ s and the transmission time by IR for 160 bits is 80ms. We compute time for different structures, with the number of neighbors for a module varying from 2 to 10. This simulation concerns our AKC-MP solution, PROLISEAN version 4 (noted V4), version 3 (noted V3), and version 1 in 128

lines of source code noted here V1(128), in 64 lines of source code V1(64) and finally in 32 lines of source code V1(32).

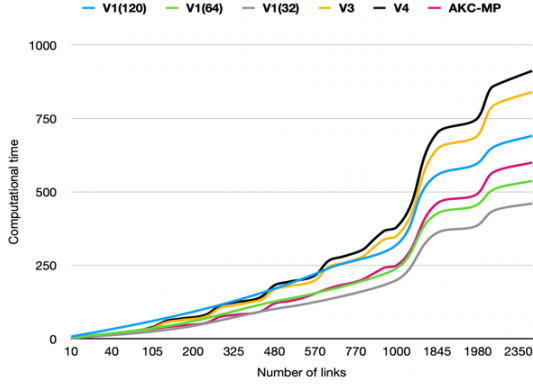


Fig. 7. Overhead computing time

For deployment, we varied the number of modules in steps of 10 up to 500, resulting in structures ranging from 10 to 500 modules. The results are shown in Fig. 7. We can observe our solution is more efficient than V3 and V4. AKC-MP becomes more expensive than V1(32) from 450 links and V1(64) from 1000 established links and remains less expensive V1(128) up to 2350 links. It has the advantage of performing only two transmissions, as transmission is a very costly operation. With two or tree neighbors per module, it achieves a lower execution time than all versions of PROLISEAN

TABLE III. OVERHEAD COMPUTATIONAL COST COMPARISONS

number of neighbors	number of links	V1 (128)	V1 (64)	V1 (64)	V3	V4	AKC-MP
2	8500	6800	4080	2720	2975	3230	2125
3	12750	7480	4760	3400	4463	4845	3188
4	17000	8160	5440	4080	5950	6460	4250
5	21250	8840	6120	4760	7438	8075	5313
6	25500	9520	6800	5440	8925	9690	6375
7	29750	10200	7480	6120	10413	11305	7438
8	34000	10880	8160	6800	11900	12920	8500
9	38250	11560	8840	7480	13388	14535	9563
10	42500	12240	9520	8160	14875	16150	10625

In a very large-scale deployments, if we vary the number of modules in steps of 10 from 10 to 500, and for each step i of 10, we scan the number of neighbors from 2 to 10, as shown in equation (4) from which we derive the average results in Table III. From equation (4), which gives the time C_4 , we can see the evolution of each solution based on the number of neighbors.

$$C_4 = \sum_{i=10}^{5000} \sum_{v=2}^{10} \frac{v \cdot m}{2} (6TH + 2Tx + 4XOR) \quad (4)$$

Thus, we see that at 34,000 links, AKC-MP is outperformed by V1(32), which has the smallest source code size, since in this case, code size becomes a critical factor for execution time. The advantage of V1 is that each module calculates the source code hash only once. Up to 42,500 links with a neighborhood size of

10, V1(128) performs no better than our solution. Versions V3 and V4 will always be less efficient than AKC-MP.

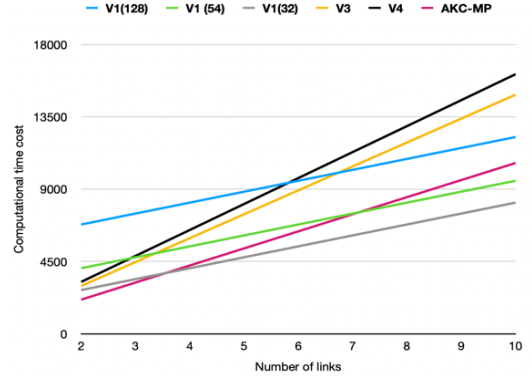


Fig. 8. Very large-scale deployment

VIII. CONCLUSION

In this paper, we have proposed a security protocol for programmable matter based modular robot. The proposed protocol implements a robust mutual authentication mechanism after which a key is generated to ensure the confidentiality of exchanged messages. The protocol uses lightweight cryptographic mechanisms to implement adaptive key generation models in a highly constrained modular robot system environment. Security analysis shows that the protocol handles many types of attack, notably replay and fabrication attacks, which are the most frequent and difficult to counter in these environments. As a follow-up to this work, we will implement this protocol with different key usage models in VisibleSim to measure their efficiency in terms of energy consumption, memory and computation of cryptographic operations. We'll also be implementing the Avispa simulator to confirm security properties.

REFERENCES

- [1] J. Bourgeois,, et al.: Programmable matter as a cyber-physical conjugation. In: 2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 002942–002947. IEEE (2016)
- [2] P., Thalamy, B. Piranda, & J. Bourgeois, (2019). A survey of autonomous self-reconfiguration methods for robot-based programmable matter. Robotics and Autonomous Systems, 2019, vol. 120, p. 103242.
- [3] Ahmadzadeh and E. Masehian, “Modular robotic systems: Methods and algorithms for abstraction, planning, control, and synchronization,” Artificial Intelligence, vol. 223, pp. 27–64, 2015.
- [4] L. Mihaylova, T. Lefebvre, H. Bruyninckx, K. Gadeyne, and J.D. Schutter, “Acomparision of decision making criteria and optimization methods for active robotic sensing,” in International Conference on Numerical Methods and Applications. Springer, 2002, pp. 316–324
- [5] J. Bassil, A. Makhoul, B. Piranda, J. Bourgeois: Distributed size-constrained clustering algorithm for modular robot-based programmable matter. ACM Trans. Auton. Adapt. Syst. 18(1), 1:1–1:21 (2023)
- [6] J. Bassil, B. Piranda, A. Makhoul, & J. Bourgeois, (2025, April). Distributed Configuration Recognition for 2D Lattice-Based Modular Robots. In : International Conference on Advanced Information Networking and Applications. Cham : Springer Nature Switzerland, 2025. p. 140-151.

- [7] Z. Derakhshandeh. 2017. Algorithmic Foundations of Self-Organizing Programmable Matter. Ph.D. Dissertation. Arizona State University.
- [8] E. Hourany, B. Habib, C. Fountaine, A. Makhoul, B. Piranda,, & Bourgeois, J. (2021). Prolisean: A new security protocol for programmable matter. *ACM Transactions on Internet Technology (TOIT)*, 2021, vol. 21, no 1, p. 1-29.
- [9] V. K. Sarker, T. N. Gia, H. Tenhunen, and T. Westerlund. 2020. Lightweight Security Algorithms for Resource-constrained ^{II}IoT-based Sensor Nodes. In *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 1–7.
- [10] M. Wazid, A. K. Das, V. Bhat, and A. V Vasilakos. 2020. LAM-CIoT: Lightweight authentication mechanism in ^{II}cloud-based IoT environment. *Journal of Network and Computer Applications* 150 (2020), 102496.
- [11] Y. Faye, H. Guyennet, I. Niang, & Y. Shou ; (2013). Fast scalar multiplication on elliptic curve cryptography in selected intervals suitable for wireless sensor networks. In : *Cyberspace Safety and Security: 5th International Symposium, CSS 2013, Zhangjiajie, China, November 13-15, 2013, Proceedings 5*. Springer International Publishing, 2013. p. 171-182
- [12] Y. Faye, I. Niang, & H. Guyennet, (2019, March). New Node Distributed Access Control Scheme for Wireless Sensor Network. In : *Proceedings of the 9th International Conference on Information Systems and Technologies*. 2019. p. 1-4.
- [13] S. Soni, & M. Hussain, (2025). Light Weight Authentication of Devices in Wireless Sensor Network. *Procedia Computer Science*, 2025, vol. 252, p. 1005-1013.
- [14] D. Saha, (2024). Improving IoT Security in Wireless Sensor Networks. *Smart Internet of Things, Smart Internet of Things*, 2024, vol. 1, no 4, p. 289-297.